App Development Project Report

App Dev Project Report

1. Student Details

Name: Akshayan Vishaal B
Roll Number: 24f2007248
Email: 24f2007248@ds.study.iitm.ac.in
About Me: I am an aspiring Computer Science engineer pursuing the IIT Madras BS Degree Program. This project provided me with an opportunity to apply the concepts learned in my coursework to a real world application. Through this experience, I strengthened my practical understanding of web technologies and backend systems.

---

2. Project Details

Project Title: Trekking Management App

Problem Statement:

This project involves building a web-based Trekking Management Application that enables Admins, Trek Staff, and Trekkers to manage trekking activities. The application provides role-based access to manage treks, bookings, participants, and trek records through a centralized system.

Approach:

The application was built using Flask as the backend framework with a modular architecture and SQLite as the database. It provides role-based access for Admins, Trek Staff, and Trekkers, enabling trek management, participant registration, booking management, and approval workflows through a centralized web interface.

---

3. AI/LLM Declaration

I used **ChatGPT** to assist in writing UI Layouts and Bootstrap enhancements. The extent of AI/LLM usage is around **20%**, limited to **code suggestions and documentation formatting**. All final implementation logic, debugging, and integration were done manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
Flask-Login	User authentication and session management
SQLite	Lightweight local database for storing user data
Werkzeug	Password hashing and security utilities
Python datetime	Handling dates and timestamps for records

---

5. Database Schema / ER Diagram

Tables:

1. **User**: Stores user account details including personal information and login credentials.
    - `(userid, name, email, password_hash, phone, role, is_active, created_at)`
2. **StaffProfile**: Stores additional information for users registered as trekking staff.
    - `(staffid, userid, experience, approval_status, joined_on)`
3. **Trek**: Stores details of trekking events including location, schedule, capacity, and assigned staff.
    - `(trekid, trekname, location, difficulty, duration, description, total_slots, available_slots, assigned_staff, status, start_date, end_date, created_at)`
4. **Booking**: Stores booking records linking users with trekking events.
    - (bookingid, userid, trekid, booking_date, booking_status)

Relationships:

- One-to-Many → `User → StaffProfile`
- One-to-Many → `User → Booking`
- One-to-Many → `StaffProfile → Trek`
- One-to-Many → `Trek → Booking`



---

6. API Resource Endpoints

Endpoint	Method	Description
`/login`	GET, POST	Authenticate user and create a session.
`/logout`	GET	Logout the current user and clear the session.
`/register`	GET, POST	Register a new trek user account.
`/staff/register`	GET, POST	Register a new trek staff account awaiting admin approval.

`/admin/dashboard`	GET	Display administrator dashboard statistics and recent bookings.
`/admin/treks`	GET	View and search all treks with filters and pagination.
`/admin/add_trek`	GET, POST	Create a new trek.
`/admin/edit_trek/<trek_id>`	GET, POST	Edit an existing trek.
`/admin/delete_trek/<trek_id>`	POST	Delete a trek.
`/admin/staff`	GET	View pending, approved and blacklisted staff members.
`/admin/update_staff_status/<staff_id>/<action>`	POST	Approve, reject, blacklist or restore a staff member.
`/admin/users`	GET	View and search registered users.
`/admin/toggle_user_status/<user_id>`	POST	Activate or deactivate a user account.
`/admin/bookings`	GET	View and search all trek bookings.
`/staff/dashboard`	GET	Display staff dashboard with assigned trek statistics.
`/staff/treks`	GET	View assigned treks with search and status filter.
`/staff/manage_trek/<trek_id>`	GET	View trek details and participants.
`/staff/update_trek_config/<trek_id>`	POST	Update available slots and trek status.
`/staff/change_status/<trek_id>/<next_state>`	POST	Mark a trek as Started or Completed.

`/staff/participants`	GET	View participants with search and filters.
`/staff/profile`	GET, POST	View and update staff profile information.
`/user/dashboard`	GET	Display user dashboard with available treks and recent bookings.
`/user/treks`	GET	Browse available treks with search, difficulty filter and pagination.
`/user/book_trek/<trek_id>`	GET, POST	View trek details and book a trek.
`/user/cancel_booking/<trek_id>`	POST	Cancel an existing booking and restore the available slot.
`/user/bookings`	GET	View user bookings and trek history with search and filters.
`/user/profile`	GET, POST	View and update user profile information.

YAML API Definition File:
Included separately in the submission ZIP as `api.yaml`.

---

7. Architecture and Features

Architecture Overview:

- **app.py** – main Flask application entry point
- **/models** – database models using SQLAlchemy
- **/controllers** – Flask Blueprints for user and activity routes
- **/templates** – Jinja2 HTML templates
- **/static** – CSS and static resources

Implemented Features:

- User, Staff, and Admin authentication with secure password hashing
- Role-based dashboards for Admin, Staff, and Trek Users.
- Trek creation, modification, deletion, and management by the administrator.

- Staff registration approval, rejection, and blacklist management.
- User trek browsing with search, filtering, and pagination.
- Trek booking and booking cancellation with automatic slot management.
- Staff management of assigned treks and participant information.
- Booking history and status tracking for users.
- Search, filtering, and pagination across multiple management pages.
- SQLite database integration using SQLAlchemy ORM for data persistence.
- Session-based authentication and authorization for secure application access.

---

8. Video Presentation

Drive Link:
24f2007248.mp4

- https://drive.google.com/file/d/1AmVojf636wV3u884BFx_N-3Lb90VfdRY/view?usp=sharing
  (Accessible to all with “View” permission.)
